

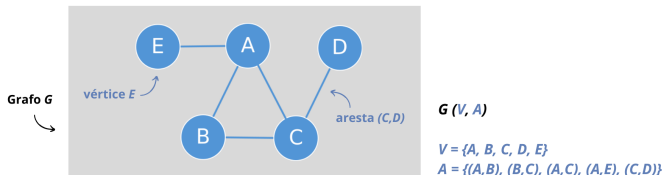
Algoritmos e Programação I

Módulo 10 - Árvore Binária

Prof^º. Elisa de Cássia Silva Rodrigues

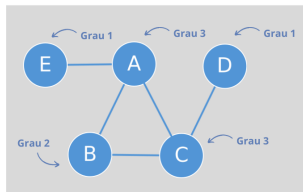
Definição:

- ▶ Modelo matemático para representar relações entre objetos de um conjunto.
- ▶ Um grafo $G(V, A)$ é definido em termos dos conjuntos:
 - ★ V : conjunto de vértices, que são itens representados no grafo.
 - ★ A : conjunto de arestas, que conectam qualquer par de vértices.

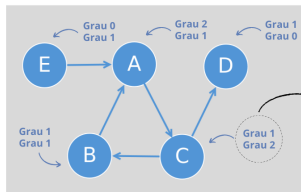


Conceitos básicos

- Grau de um vértice:



Grafo (não direcionado)

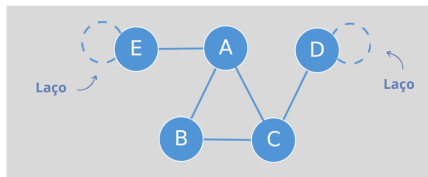


Digrafo (direcionado)



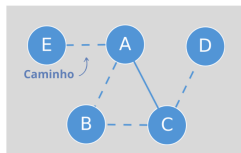
Conceitos básicos

- Laços:

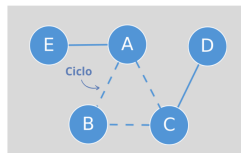


Conceitos básicos

- Caminhos e ciclos:

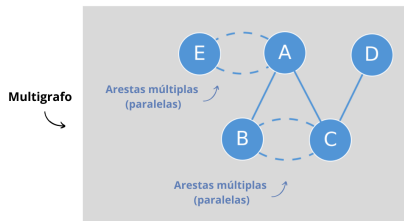


Caminho: *E-A-B-C-D*



Ciclo: *A-B-C*

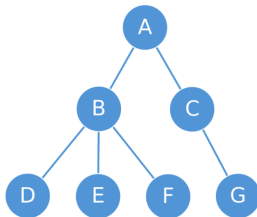
- Arestas múltiplas e multigrafo:



Árvore

- Definição:

- ▶ Abstração matemática usada para representar estruturas hierárquicas não lineares.
- ▶ Tipo especial de grafo (não direcionado, conexo e acíclico) representado por um conjunto de nós ou nodos (vértices) e arestas (conexões).

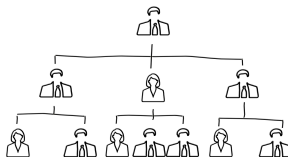


● Aplicações:

- ▶ Estrutura de diretórios do computador.
- ▶ Diagrama hierárquico de uma organização.
- ▶ Campeonatos de modalidades desportivas.
- ▶ Relações de dependência (árvore genealógica).
- ▶ Busca de dados.

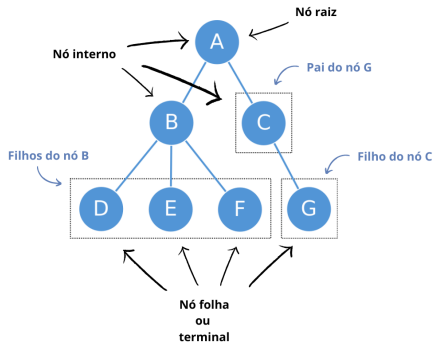


Fonte: <https://commons.wikimedia.org/wiki/File:FileAndFolders.png>



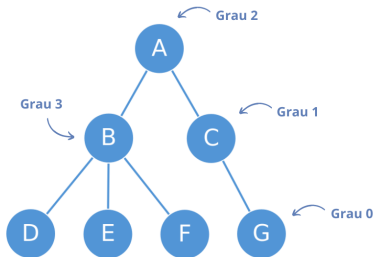
Conceitos básicos

- Notação:



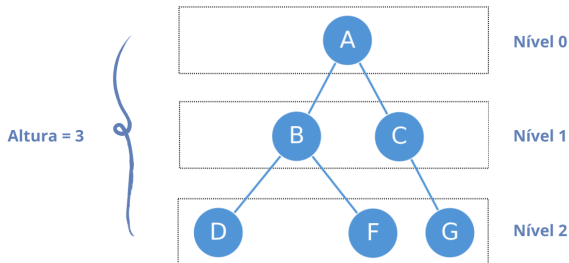
Conceitos básicos

- Grau do nó e subárvores:



Conceitos básicos

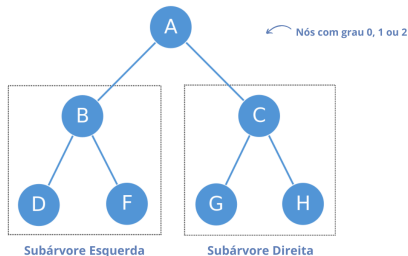
- Altura e nível da árvore:



Árvore Binária

- Definição:

- ▶ Tipo especial de **árvore**, onde cada nó pode possuir no máximo **duas** subárvores (esquerda e direita).
- ▶ Se um nó possui nenhuma subárvore, então este é um **nó folha**.



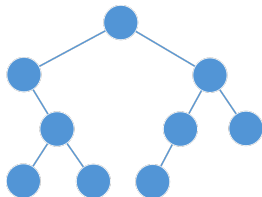
Tipos de Árvore Binária

- Estritamente Binária:

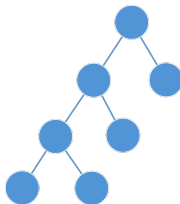
- ▶ Todo nó possui duas subárvores ou nenhuma.

- Completa:

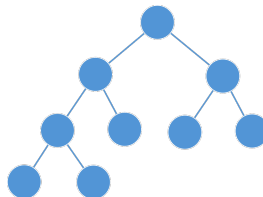
- ▶ Todos os nós folha estão no último ou no penúltimo nível.



Árvore Binária



Árvore Estritamente Binária

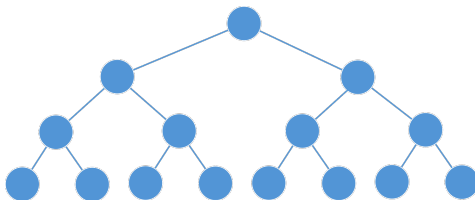


Árvore Completa

Tipos de Árvore Binária

- Cheia:

- ▶ Árvore estritamente binária com todos os nós folha no último nível.
- ▶ Considerando que o nó **raiz** fica no **nível 0**, temos que:
 - ★ O nível n possui 2^n nós.
 - ★ Uma árvore com altura h possui $2^h - 1$ nós.
 - ★ Se o nível n possui m nós, então o nível $n + 1$ possui $2m$ nós.



Árvore Cheia

- Operações básicas:

- ▶ Criação da árvore.
- ▶ Inserção de um elemento.
- ▶ Remoção de um elemento.
- ▶ Busca por um elemento.
- ▶ Destruição da árvore.
- ▶ Informação sobre número de nós.
- ▶ Informação sobre altura da árvore.
- ▶ Informação sobre a árvore estar vazia ou cheia.

A implementação das operações de uma árvore binária depende do tipo de alocação de memória usada (estática ou dinâmica).

Árvore Binária Estática

- Definição:

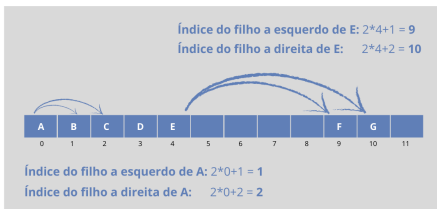
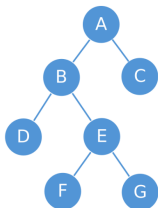
- ▶ **Árvore binária** definida utilizando **alocação estática** e **acesso sequencial** dos elementos.

- Características:

- ▶ Definida por um **vetor** com elementos sequenciais na memória.
- ▶ Exige a definição prévia do número máximo de elementos (**MAX**).

Árvore Binária Estática

- Em um vetor que representa uma árvore binária, cada elemento é considerado pai dos elementos nas duas próximas posições livres.
- Dada o índice p de um nó da árvore, temos que:
 - O índice do **filho da esquerda** de p é $2p + 1$.
 - O índice do **filho da direita** de p é $2p + 2$.



Árvore Binária Encadeada

- Definição:

- ▶ **Árvore binária** definida utilizando **alocação dinâmica** e **acesso encadeado** dos elementos.

- Características:

- ▶ Existe um ponteiro que indica a **raiz** da árvore.
- ▶ Cada elemento possui um **dado** e dois ponteiros para:
 - ★ Subárvore esquerda: **esq.**
 - ★ Subárvore direita: **dir.**
- ▶ Cada elemento é alocado dinamicamente quando é inserido na árvore.
- ▶ Se um elemento é removido, a memória alocada para ele é liberada.

Árvore Binária Encadeada

- Definição do TAD Árvore Binária Encadeada:

- ▶ Definir os arquivos `arvoreBinaria.h` e `arvoreBinaria.c`.
- ▶ Declarar o tipo de dado que irá representar a árvore no `arquivo .h`:
- ▶ Definir o tipo de dado que será armazenado dentro da árvore (`int`).
- ▶ Declarar a estrutura para representar a árvore no `arquivo .c`:

```
typedef struct no* ArvoreBin;  
  
struct no{  
    int dado;  
    struct no *esq;  
    struct no *dir;  
};  
  
typedef struct no No;
```

- Declarar um ponteiro do tipo `ArvoreBin` para acessar o TAD (`main.c`):

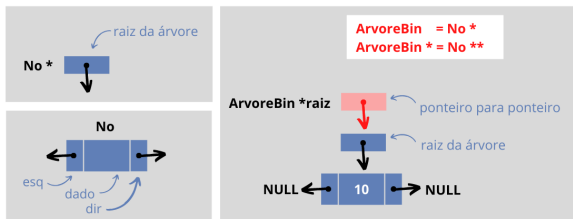
```
ArvoreBin *raiz;
```

Árvore Binária Encadeada

- Ilustração dos tipos de dados **Árvore Binária** e **No**:

```
typedef struct no No;
```

```
typedef struct no* ArvoreBin;
```



Note que, nesta implementação, a estrutura **ArvoreBin** é abstrata (ponteiro para **No**).

Árvore Binária Encadeada

- Criação da árvore binária:

- ▶ Antes de usar uma árvore é preciso criar uma **árvore vazia**.
- ▶ Isto é, alocar um espaço na memória para o ponteiro **raiz**:
 - ★ Alocação dinâmica de um ponteiro do tipo **ArvoreBin** usando **malloc()**.
- ▶ A árvore está vazia, quando **raiz != NULL** e ***raiz == NULL**.

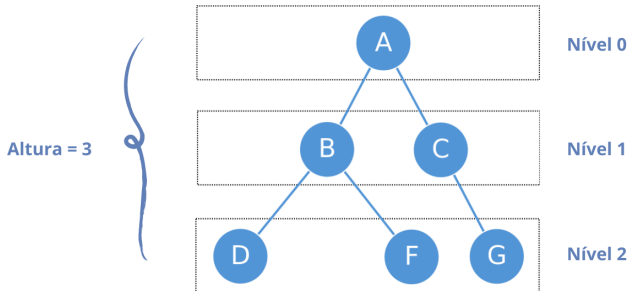
Árvore Binária Encadeada

- Destruição da árvore binária:

- ▶ Inicialmente, deve-se liberar a memória alocada para todos os elementos da árvore:
 - ★ Liberação da estrutura **No** usando **free()**.
- ▶ Deve-se liberar a memória alocada para o ponteiro da raiz da árvore:
 - ★ Liberação do ponteiro do tipo **ArvoreBin** usando **free()**.

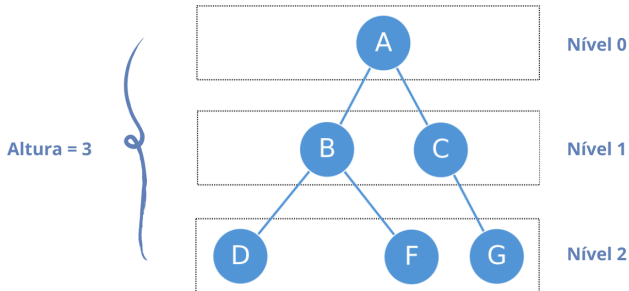
Árvore Binária Encadeada

- Número total de nós (função recursiva).
 - ▶ Exemplo: árvore com 6 nós.



Árvore Binária Encadeada

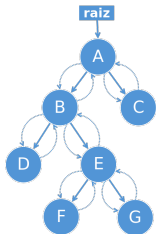
- **Altura da árvore** (função recursiva):
 - ▶ Exemplo: árvore com altura 3.



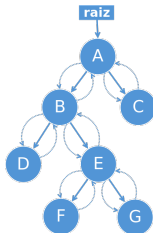
Árvore Binária Encadeada

- Percorrendo uma árvore (recursivamente):

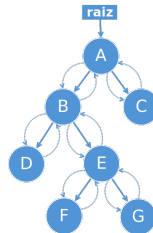
- ▶ Não existe uma forma natural para percorrer uma árvore a fim de executar alguma ação sobre todos os nós.
- ▶ As formas mais utilizadas para percorrer um árvore binária são:
 - ★ **Percurso pré-ordem:** visita a raiz, o filho da esquerda e o filho da direita.
 - ★ **Percurso em ordem:** visita o filho da esquerda, a raiz e o filho da direita.
 - ★ **Percurso pós-ordem:** visita o filho da esquerda, o filho da direita e a raiz.



Percurso pré-ordem: ABDEFGC



Percurso em ordem: DBFEGAC



Percurso pós-ordem: DFGEBCA

Árvore Binária Encadeada

As operações de **inserção** e **remoção** em uma árvore binária dependem da aplicação.

Como aplicação, veremos a **Árvore Binária de Busca** e suas funções para inserção e remoção de elementos.

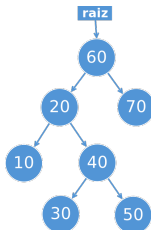
Árvore Binária de Busca

- Definição:

- ▶ Tipo especial de **árvore binária** onde cada nó possui um valor (chave) associado a ele que determina a posição do nó na árvore.

- Características:

- ▶ Todos os valores da subárvore esquerda são menores do que o nó pai.
- ▶ Todos os valores da subárvore direita são maiores do que o nó pai.
- ▶ Inserções e remoções devem respeitar as regras acima.



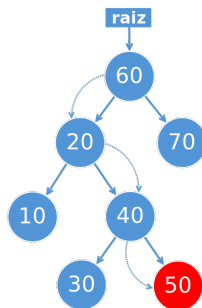
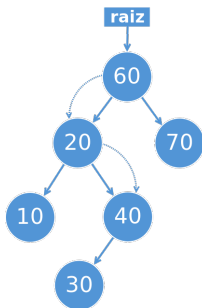
Árvore Binária de Busca

● Inserção de um nó:

- ▶ Envolve a criação de um novo nó:
 - ★ Verificar se a árvore existe (**raiz** **!= NULL**).
 - ★ Alocar memória dinamicamente para o novo nó.
 - ★ Atribuir o valor do novo nó ao campo **dado**.
 - ★ Os ponteiros **esq** e **dir** do novo nó apontam para **NULL**.
- ▶ Envolve a busca pela posição de inserção:
 - ★ Comparar o valor a ser inserido com a raiz.
 - ★ Se o valor for menor, vai para a subárvore esquerda.
 - ★ Se o valor for maior, vai para a subárvore direita.
 - ★ Aplicar o método recursivamente até chegar a um nó nulo.
- ▶ Se o valor do novo nó for menor que o valor do último nó visitado, o ponteiro **esq** desse nó aponta o novo nó.
- ▶ Se o valor do novo nó for maior que o valor do último nó visitado, o ponteiro **dir** desse nó aponta o novo nó.

Árvore Binária de Busca

- Exemplo de inserção: 50.



● Remoção de um nó:

- ▶ Envolve a liberação da memória alocada para o nó a ser excluído:
 - ★ Verificar se a árvore existe (`raiz != NULL`).
 - ★ Verificar se a árvore não está vazia (`*raiz != NULL`).
- ▶ Envolve a busca pelo nó a ser removido:
 - ★ Comparar o valor a ser removido com a raiz.
 - ★ Se o valor for igual, remove o nó.
 - ★ Se o valor for menor, vai para a subárvore esquerda.
 - ★ Se o valor for maior, vai para a subárvore direita.
 - ★ Aplicar o método recursivamente até chegar ao nó desejado.

Árvore Binária de Busca

• Remoção de um nó:

- ▶ Liberar a memória do nó **atual** usando a função **free()**.
- ▶ Se o nó a ser removido for um nó folha, basta liberar a memória.
- ▶ Se for um nó interno, é preciso reorganizar a árvore antes da remoção.
- ▶ Para isso, é criado um ponteiro auxiliar **ant** que deve apontar para o nó pai do nó a ser removido (**atual**).
- ▶ Se **atual** for um nó interno (sem filho a esquerda):
 - ★ O filho a direita do nó **atual** passa a ser filho do nó **ant**, substituindo o nó **atual** que será removido.

Árvore Binária de Busca

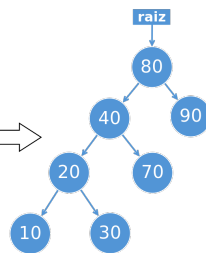
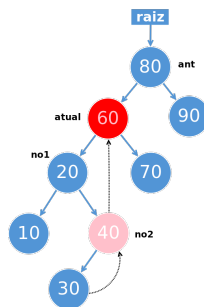
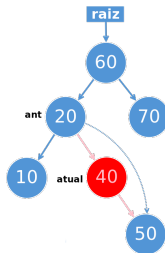
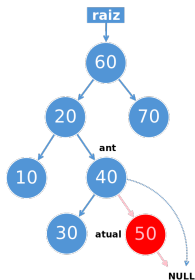
- Remoção de um nó:

- ▶ Se **atual** for um nó interno (com filho a esquerda):

- ★ Percorrer a subárvore da esquerda do nó **atual**.
- ★ Encontrar o nó folha mais a direita (**no2**).
- ★ Se o nó pai do **no2** (**no1**) for diferente do **atual**:
 - O filho da esquerda do **no2** passa a ser filho da direita do **no1**.
 - O filho da esquerda do **atual** passa a ser filho da esquerda do **no2**.
- ★ O filho da direita do **atual** passa a ser o filho da direita do nó **no2**.
- ★ Por fim, o **no2** passa a ser filho do nó **ant**, substituindo o nó **atual** que será removido.

Árvore Binária de Busca

- Três exemplos de remoção: 50, 40 e 60.

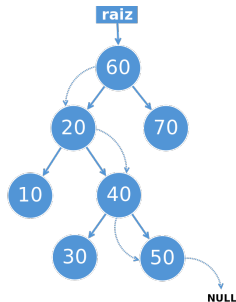
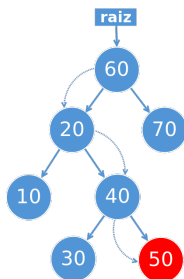


Árvore Binária de Busca

- Consulta a um nó:
 - ▶ Envolve a busca pelo nó desejado (**atual**):
 - ★ Comparar o valor buscado com a raiz.
 - ★ Se o valor for igual, indica que o valor está na árvore.
 - ★ Se o valor for menor, vai para a subárvore esquerda.
 - ★ Se o valor for maior, vai para a subárvore direita.
 - ★ Aplicar o método recursivamente até chegar ao nó desejado.
 - ★ Se o nó **atual == NULL**, o valor não existe na árvore.

Árvore Binária de Busca

- Dois exemplos de consulta: 50 (encontrado) e 55 (não encontrado).



Tipos de Árvore Binária de Busca

- **Árvore binária de busca balanceada:**
 - ▶ Árvore binária de busca onde as alturas das subárvores esquerda e direita de cada nó da árvore tem uma diferença máxima de uma unidade (fator de balanceamento).
- **Exemplos:**
 - ▶ Árvore AVL.
 - ▶ Árvore rubro-negra.
 - ▶ Árvore B, B+ e B*.

Tópicos a serem estudados na próxima disciplina de
Algoritmo e Estrutura de Dados II (COM112).

Implementação

- Implementação TAD Árvore Binária Encadeada:

https://repl.it/@elisa_rodrigues/Modulo10-ArvoreBinaria

- Exercício para fixação:

Implemente a TAD Árvore Binária Estática.

- ① BACKES, A. *Estrutura de dados descomplicada em linguagem C*. 2016.

-> **Capítulo 10: Grafos**

-> Material Complementar - Vídeo aulas (56ª a 59ª).

-> **Capítulo 11: Árvores**

-> Material Complementar - Vídeo aulas (67ª a 77ª).

<https://programacaodescomplicada.wordpress.com/indice/estrutura-de-dados/>